Chapter 12 — Transitions, Transforms and Animation

Motion, used well, explains what just happened: a menu slides out of its button, a card grows into the detail view, a spinner says "working". Used badly it distracts, makes some people physically ill, and causes **jank** (visible stutter when the browser misses frames). CSS has three core tools — **transforms** (move, rotate, scale without touching layout), **transitions** (animate a change between states) and **keyframe animations** (a timeline of states) — plus newer ones for entering/leaving elements, **view transitions** and **scroll-driven** animation. This chapter shows how the browser computes every in-between frame, then covers performance and accessibility.

In this chapter: 12.1 Transitions · 12.2 Timing functions · 12.3 2D transforms · 12.4 3D transforms · 12.5 Keyframe animations · 12.6 The Web Animations API · 12.7 Performance · 12.8 Enter and exit animations (`@starting-style`, `allow-discrete`) · 12.9 Reduced motion and accessibility · 12.10 View transitions · 12.11 Scroll-driven animations.

How the figures and numbers were made. Every example is a page in `code/`. `code/make_figures.mjs` makes **filmstrips**: it pauses every animation with the Web Animations API (12.6; CSS transitions and animations appear as `CSSTransition` / `CSSAnimation` objects in `document.getAnimations()`), sets `currentTime` to an exact moment and screenshots — so each frame is exactly t = 0, 200, 400 … ms. The same trick in `code/anim.mjs` lets the `NN_*.mjs` scripts (Node + Puppeteer driving headless Chrome) read **computed values mid-animation** and compare them with the maths; measurement uses `code/measure.mjs` (Chapter 10's helper).

```
// code/anim.mjs — freeze every animation at the same moment
export const seek = (page, ms) => page.evaluate(ms => {
  for (const a of document.getAnimations()) { a.pause(); a.currentTime = ms; }
  return new Promise(r => requestAnimationFrame(() => requestAnimationFrame(r)));   // let the frame render
}, ms);
```

12.1 Transitions

What it is

A **transition** says: "when this property's value changes, move smoothly from old to new over this time". Something else changes the value (`:hover` / `:focus-visible`, a JS-added class, a media query); the transition only animates the change.

```
.button {
  background: #2563eb;
  transition: background-color 200ms ease-out, translate 150ms ease-out;
}
.button:hover { background: #1d4ed8; translate: 0 -2px; }
```

How it works

1. The browser computes the style before and after the change.
2. For each `transition-property` whose value changed **and can be interpolated** (sensible in-between values exist, as for 10px → 20px), it creates a transition.
3. Each **frame** (one screen redraw, typically 60 per second) it computes *progress* = timing-function(elapsed / duration) and sets old + (new − old) × progress.
4. If the value changes again mid-way, the transition is replaced by one starting from the **current** value; reversing a half-finished transition is **shortened** proportionally (un-hovering after 50ms doesn't take the full duration).

The properties

Longhand	Example	Meaning
`transition-property`	`opacity, translate` | `all`	What to animate (`all` = every animatable property — convenient but can animate things you didn't intend)
`transition-duration`	`200ms`	How long (default `0s` = no transition)
`transition-timing-function`	`ease-out`	Speed curve (12.2); default `ease`
`transition-delay`	`100ms`	Wait before starting (negative = start part-way)
`transition-behavior`	`allow-discrete`	Also transition **discrete** properties like `display` (12.8)
`transition`	`opacity 200ms ease-out 50ms`	Shorthand; first time = duration, second = delay; comma-separate several

What can be animated

- **Interpolable**: lengths, percentages, numbers, colours, transforms, shadows, gradient colour stops (not whole gradients in most browsers), `grid-template-columns` track sizes (Baseline since 2022), `clip-path` shapes with equal point counts…
- **Discrete** (`display`, `visibility`, `font-family`, `position`…) can't be in-betweened: normally they flip; with `transition-behavior: allow-discrete` they flip at 50 % — except `display`/`content-visibility`, which stay visible for the whole transition (12.8).
- `auto` isn't interpolable: `height: 0 → auto` jumps. Chromium's `interpolate-size: allow-keywords` / `calc-size()` fix it but are **not Baseline** (§1.1 defines Baseline); portable options: animate `grid-template-rows: 0fr → 1fr` on a wrapper, or measure the height in JS.

Events

`transitionrun`, `transitionstart` (after the delay), `transitionend` (once **per property** — two for `opacity .2s, translate .2s`), `transitioncancel` (interrupted). Filter by `event.propertyName` so a `background-color` `transitionend` doesn't close your menu early.

Common mistakes

- `transition` only in the `:hover` rule — hover-in animates, hover-out is instant. Put it on the base rule.
- `transition: all 300ms` where layout properties change — animates widths and paddings you never meant to (expensively, 12.7).
- Expecting a transition when an element is **added** or leaves `display: none` — no "before" style exists. `@starting-style` solves it (12.8).
- Too long: UI feedback feels right around **100–300ms**; larger movements up to ~500ms.

12.2 Timing functions (easing)

What it is

A **timing function** (*easing*) maps **time progress** (0 → 1) to **output progress** (usually 0 → 1). Linear motion looks mechanical; real objects accelerate and decelerate.

Function	Shape	Typical use
`linear`	straight line	spinners, progress, colour cycling
`ease` (default)	quick start, long gentle stop	general purpose
`ease-in`	slow start, fast end	things **leaving** the screen
`ease-out`	fast start, slow end	things **entering** or responding to input (snappy)
`ease-in-out`	slow–fast–slow	moving between two on-screen positions
`cubic-bezier(x1, y1, x2, y2)`	any smooth curve; y may leave 0–1 for overshoot	brand motion, "back" effects
`steps(n, jump-start | jump-end | jump-none | jump-both)`	n discrete jumps	sprite sheets (one image holding every frame), typewriter, ticking clocks
`linear(0, 0.36 30%, 1 45%, …)`	straight lines through listed points	bounces and springs (Baseline since December 2023)

Keywords are shortcuts: `ease` = `cubic-bezier(.25, .1, .25, 1)`, `ease-in` = `(.42, 0, 1, 1)`, `ease-out` = `(0, 0, .58, 1)`, `ease-in-out` = `(.42, 0, .58, 1)`; `step-start` = `steps(1, jump-start)`, `step-end` = `steps(1, jump-end)`.

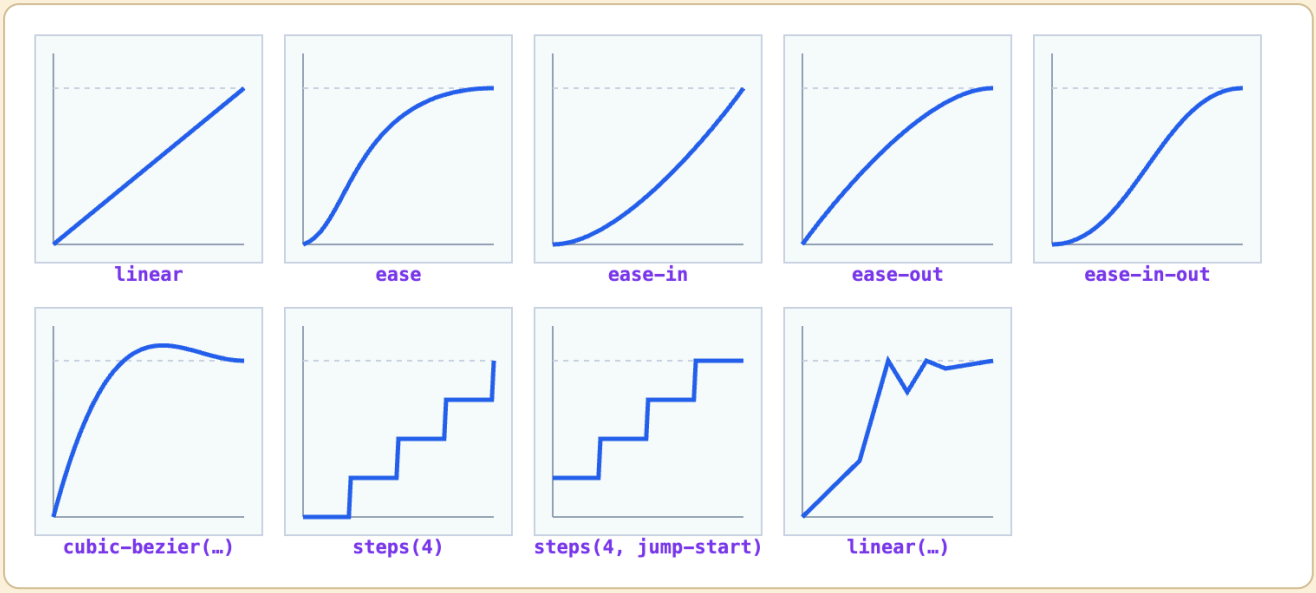


Figure 12.1 — `code/02_curves.html` animates a probe per easing, pauses it at 101 moments and reads the computed value — Chrome's real curves. Dashed line = 100 %.

How a cubic Bézier is computed

`cubic-bezier(x1, y1, x2, y2)` is a curve from (0, 0) to (1, 1) with two control points, both coordinates functions of a parameter *s*: `x(s) = 3·x1·s·(1−s)² + 3·x2·s²·(1−s) + s³`, likewise `y` with `y1`, `y2`. For time *t* the browser **solves x(s) = t** for *s* — x is time, so it must stay within 0–1 to give one answer; y is output and may overshoot — then returns y(s). `code/01_timing.mjs` does this by **bisection** (halve the range containing the answer until close enough) and compares with the browser.

The proof: a race

`code/01_timing.html` transitions eight balls 160px over 1000ms, each with a different easing:

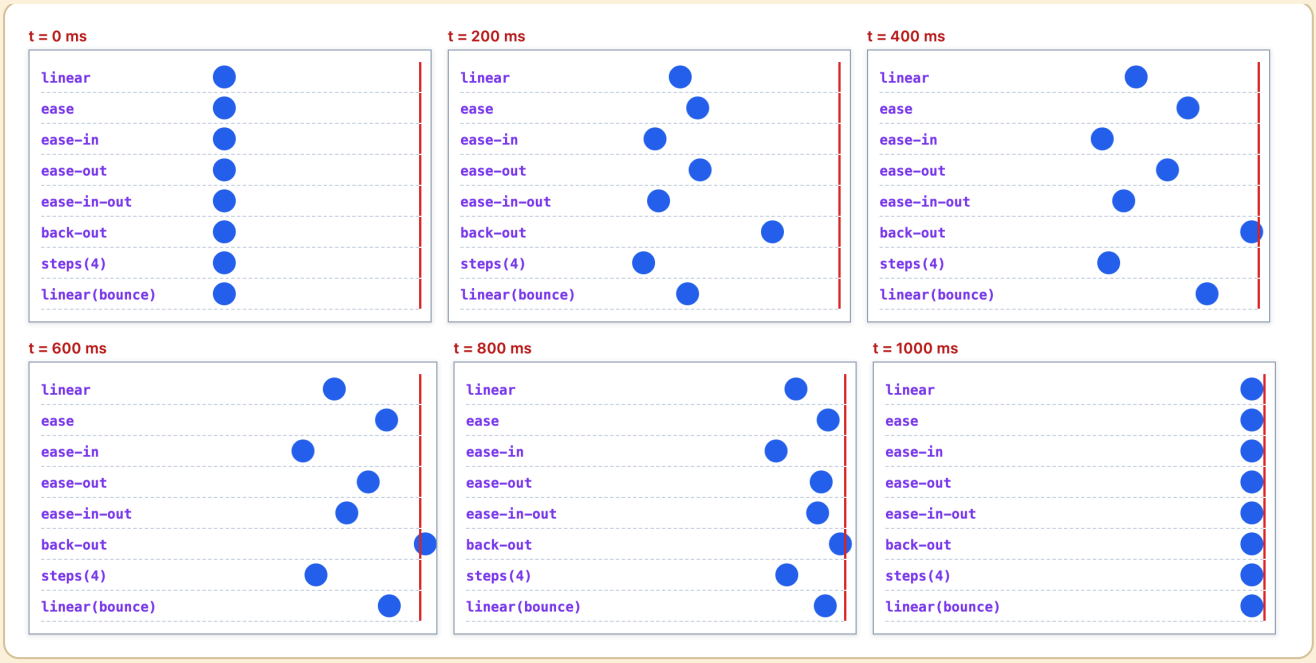


Figure 12.2 — Filmstrip made by pausing all eight transitions and seeking them to each time.

Browser values (the script prints a second "our maths" row under each; every one is identical to three decimals, so they're merged here):

```
progress (0 = start, 1 = end) at t =        0ms   200ms   400ms   600ms   800ms  1000ms
linear          browser       0.000   0.200   0.400   0.600   0.800   1.000
ease            browser       0.000   0.295   0.683   0.885   0.976   1.000
ease-in         browser       0.000   0.062   0.215   0.429   0.692   1.000
ease-out        browser       0.000   0.308   0.571   0.785   0.938   1.000
ease-in-out     browser       0.000   0.082   0.332   0.668   0.918   1.000
back-out        browser       0.000   0.703   1.030   1.097   1.042   1.000
steps(4)        browser       0.000   0.000   0.250   0.500   0.750   1.000
linear(bounce)  browser       0.000   0.240   0.787   0.900   0.960   1.000
```

- `ease` covers 68 % of the distance at 40 % of the time — why it feels quick.
- `back-out` = `cubic-bezier(.34, 1.56, .64, 1)`: y1 = 1.56 > 1, so progress overshoots (109.7 % at 600ms) and settles.
- `steps(4)` defaults to `jump-end`: the value changes at the *end* of each quarter, so at 200ms it's still 0 (`steps(2)` over 1s → 0.5 at 0.6s).
- `linear(0, 0.36 30%, 1 45%, 0.8 55%, …)`: interpolates linearly between points; at 600ms (between 55 % → 0.8 and 65 % → 1) it's 0.9.

Choosing

Entering/responding: `ease-out` (or a `cubic-bezier(.2, .8, .2, 1)`-style curve). Leaving: `ease-in`, shorter than the entrance. On-screen moves: `ease-in-out`. Continuous rotation: `linear`. Design systems define a few named easing/duration **tokens** (`--ease-out`, `--dur-fast: 150ms`) and use only those.

12.3 2D transforms

What it is

`transform` changes where an element is **painted** — moved, rotated, scaled, skewed — **without changing its layout box**: neighbours don't move (so a hover `translate: 0 -4px` never shifts the elements below), and the flow space stays reserved. That's what makes transforms ideal for animation.

Function	Example	Effect
`translate(x, y)` / `translateX()` / `translateY()`	`translate(40px, 20px)`	Move; `%` is relative to the element's **own** size — `top: 50%; left: 50%; translate: -50% -50%` centres an absolutely positioned element of unknown size
`rotate(angle)`	`rotate(30deg)`, `0.25turn`	Clockwise rotation
`scale(sx, sy)`	`scale(1.4, 0.6)`	Resize (1 = unchanged; negative flips)
`skew(ax, ay)` / `skewX()`	`skewX(-20deg)`	Slant
`matrix(a, b, c, d, e, f)`	—	The general form everything reduces to

`transform-origin` (default `50% 50%`, the centre) is the fixed point for rotate, scale and skew.

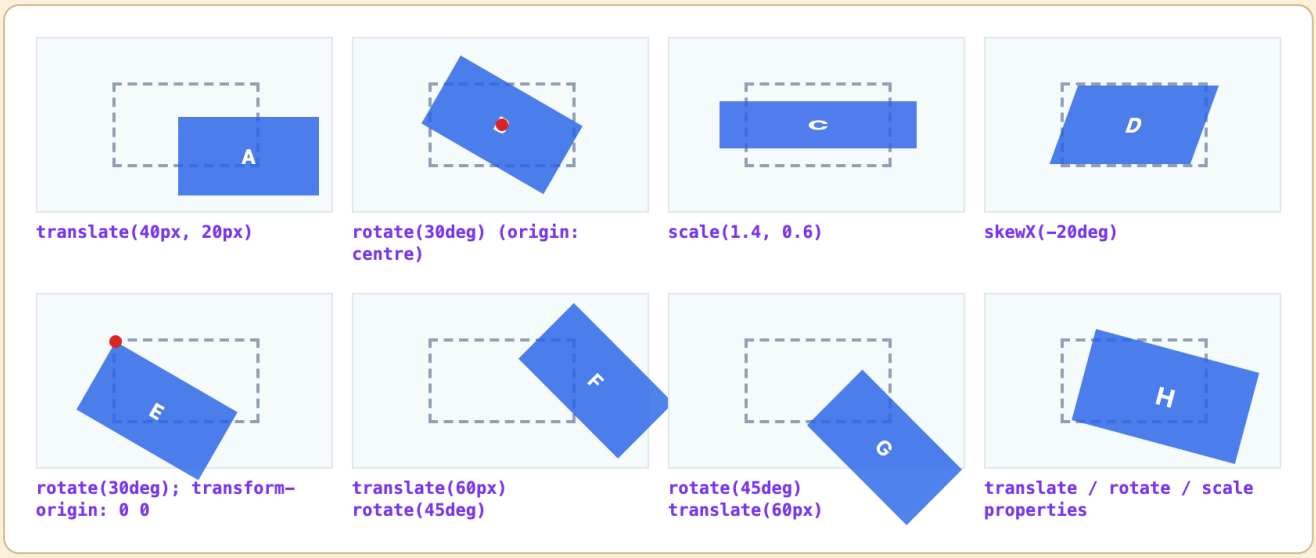


Figure 12.3 — `code/03_transforms.html`. Red dots mark the transform origin.

Order matters

`translate(60px, 0) rotate(45deg)` (F) and `rotate(45deg) translate(60px, 0)` (G) differ:

- Read **right to left** as operations on the element: F = rotate, then move right 60px.
- Or **left to right** as moving the **coordinate system**: G = rotate the axes 45°, then move 60px along the *rotated* x-axis — diagonally. (So `rotate(90deg) translateX(100px)` moves the element *down*.)

`code/03_transforms.mjs` prints the layout box (unchanged for all), the painted **bounding box** (smallest upright rectangle enclosing the drawn shape) and the resolved matrix:

```
A t1: layout 50,30 90x50 | painted bbox 90,50 90x50 | transform matrix(1, 0, 0, 1, 40, 20)
B t2: layout 50,30 90x50 | painted bbox 43.5,10.8 102.9x88.3 | transform matrix(0.866025, 0.5, -0.5, 0.866025, 0,
0)
C t3: layout 50,30 90x50 | painted bbox 32,40 126x30 | transform matrix(1.4, 0, 0, 0.6, 0, 0)
D t4: layout 50,30 90x50 | painted bbox 40.9,30 108.2x50 | transform matrix(1, 0, -0.36397, 1, 0, 0)
E t5: layout 50,30 90x50 | painted bbox 25,30 102.9x88.3 | transform matrix(0.866025, 0.5, -0.5, 0.866025, 0, 0)
F t6: layout 50,30 90x50 | painted bbox 105.5,5.5 99x99 | transform matrix(0.707107, 0.707107, -0.707107,
0.707107, 60, 0)
G t7: layout 50,30 90x50 | painted bbox 87.9,47.9 99x99 | transform matrix(0.707107, 0.707107, -0.707107,
0.707107, 42.4264, 42.4264)
H t8: layout 50,30 90x50 | painted bbox 55.1,22 119.8x85.9 | transform none
H individual properties: translate 20px 10px, rotate 15deg, scale 1.2 (transform stays none)
```

Check B: a 90×50 box rotated 30° has a bounding box 90·cos30° + 50·sin30° = 77.9 + 25 = **102.9** wide and 90·sin30° + 50·cos30° = 45 + 43.3 = **88.3** tall ✓. In `matrix(a, b, c, d, e, f)`, (a, b, c, d) = (cos θ, sin θ, −sin θ, cos θ) for a rotation and (e, f) is the translation: G's is 60·cos45° = **42.43** in x and y — the 60px move along the rotated axis ✓. B and E share a matrix; only `transform-origin` differs.

Individual transform properties

`translate`, `rotate` and `scale` are also **separate properties** (Baseline since 2022, widely available), applied in a fixed order — translate, rotate, scale, then `transform` — so order surprises can't happen, and each animates **independently** (a hover `scale` doesn't clobber an animation's `translate`). They don't appear in `transform` (H shows `none`). This chapter's pages use them wherever possible.

Side effects

- A non-`none` transform creates a **stacking context** and becomes the **containing block** for `position: fixed` and `absolute` descendants (§7.6, §7.8).
- Transforms don't apply to non-replaced **inline** boxes (a plain `<span>`); make it `inline-block`.
- Scaled-up text can look blurry mid-animation on some GPUs; animate to a crisp final state.
- `getBoundingClientRect()` returns the **painted** box (transform included); `offsetWidth`/`offsetLeft` the layout box.

12.4 3D transforms

What it is

3D adds a **z-axis** toward the viewer: `translateZ()`, `rotateX()`, `rotateY()`, `rotate3d()`, `scale3d()`, `matrix3d()`, and `rotate: y 60deg`. To look 3D they need **perspective**.

Property	On	Meaning
`perspective: 600px`	the **parent** (scene)	Viewer's distance to the z = 0 plane; smaller = stronger distortion. All children share one vanishing point.
`perspective()` function	the element's own `transform`	Per-element perspective (each has its own vanishing point)
`perspective-origin`	the parent	Where the viewer's eye is (default centre)
`transform-style: preserve-3d`	a parent whose children stay in 3D	Otherwise children are flattened into its plane
`backface-visibility: hidden`	a face	Hide the element when its back faces the viewer

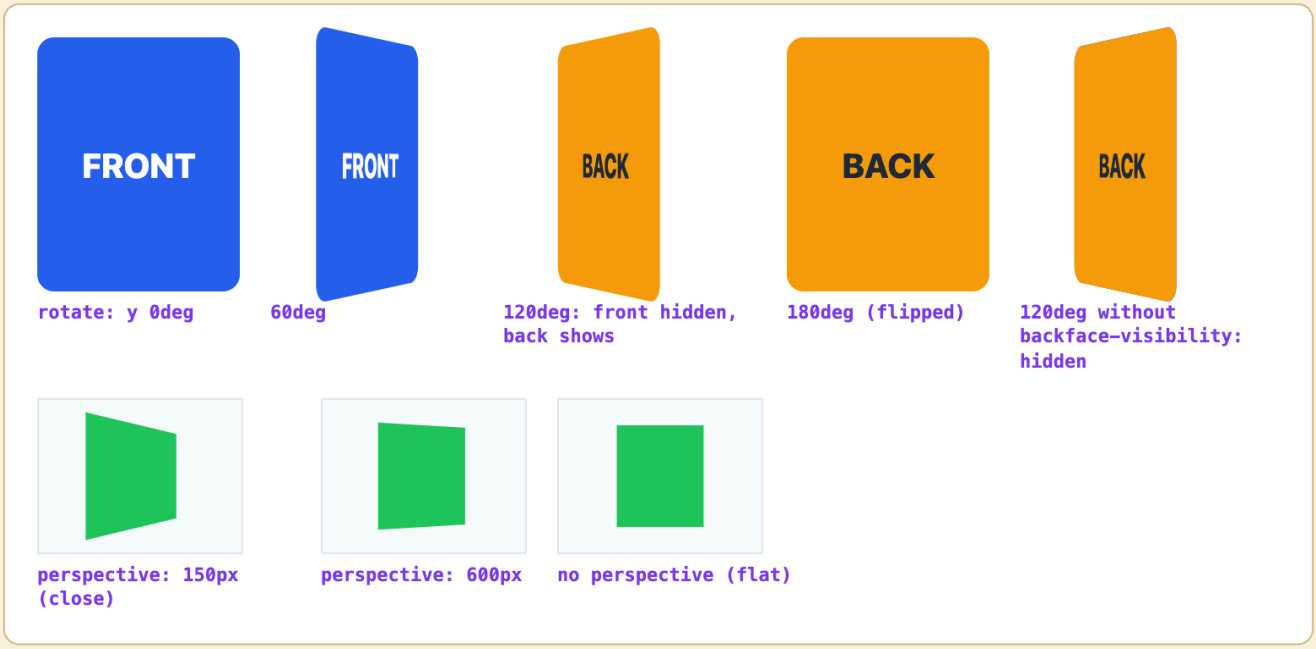


Figure 12.4 — `code/04_3d.html`.

The flip card, step by step

```
.scene { perspective: 600px; }                                  /* 1. give the scene depth */
.card  { position: relative; transform-style: preserve-3d;       /* 2. keep both faces in 3D */
         transition: rotate 600ms ease-in-out; }
.face  { position: absolute; inset: 0; backface-visibility: hidden; }   /* 3. hide a face pointing away */
.back  { rotate: y 180deg; }                                    /* 4. the back starts facing away */
.scene:hover .card, .card.flipped { rotate: y 180deg; }         /* 5. flip the whole card */
```

Past 90° the front points away (hidden) and the pre-rotated back faces the viewer — hence `backface-visibility: hidden` on both faces. In a real component flip on a **button** (toggle `.flipped`), not only hover, so keyboard and touch users can use it.

Things that flatten 3D

`overflow` other than `visible`, `opacity < 1`, `filter`, `clip-path`, `mask`, `mix-blend-mode`, `isolation: isolate` on the `preserve-3d` element render its children flat. If your cube collapses, look for one of these.

12.5 Keyframe animations

What it is

A **keyframe animation** runs a named sequence of states independent of any state change — it can start on load, loop, alternate and be delayed.

```
@keyframes bounce {
  0%, 100% { translate: 0 -12px; animation-timing-function: ease-in; }   /* falling: speed up   */
  50%      { translate: 0 12px;  animation-timing-function: ease-out; }  /* rising: slow down   */
}
.ball { animation: bounce 1000ms infinite; }

@keyframes spin { to { rotate: 1turn; } }        /* missing "from" = the element's own current value */
.spinner { animation: spin 1s linear infinite; }
```

The `animation-*` properties

Longhand	Values	Notes
`animation-name`	`bounce`	The `@keyframes` name
`animation-duration`	`1s`	One iteration
`animation-timing-function`	`ease` …	Applies **between each pair of keyframes**, not across the whole animation; overridable per keyframe (as in `bounce`)
`animation-delay`	`500ms`, `-250ms`	A **negative** delay starts part-way — handy for staggering spinners
`animation-iteration-count`	`3`, `infinite`, `0.5`	
`animation-direction`	`normal` | `reverse` | `alternate` | `alternate-reverse`	`alternate` plays every other iteration backwards
`animation-fill-mode`	`none` | `forwards` | `backwards` | `both`	What applies **before** (during the delay) and **after**
`animation-play-state`	`running` | `paused`	Pause/resume from CSS
`animation-composition`	`replace` | `add` | `accumulate`	Combining with the underlying value (Baseline 2023)
`animation-timeline`	`auto` | `scroll()` | `view()`	Drive by scroll instead of time (12.11)
`animation`	`bounce 1s ease-in-out 200ms infinite alternate both`	Shorthand; first time = duration, second = delay (`pulse 2s 500ms` → 500ms delay)

Keyframe rules: `from` = `0%`, `to` = `100%`; a missing 0 %/100 % uses the element's normal value; percentages can be grouped (`0%, 100%`); `!important` inside keyframes is ignored. While running, animation values beat normal declarations but lose to `!important` ones (§6.5).

The proof: values mid-animation

`code/05_keyframes.html` runs a bounce, a spinner, an alternating slide and four copies of a colour animation (`red → green`, 500ms, **`after a 500ms delay`**) with different fill modes; `code/05_keyframes.mjs` seeks them:

```
t=   0ms bounce 0px -12px spin 0deg     slide 0px       | fill none=grey (own style), forwards=grey (own style),
backwards=red, both=red
t= 250ms bounce 0px -4.43144px spin 90deg    slide 20.6659px | fill none=grey (own style), forwards=grey (own
style), backwards=red, both=red
t= 499ms bounce 0px 11.9179px spin 179.64deg slide 79.7241px | fill none=grey (own style), forwards=grey (own
style), backwards=red, both=red
t= 750ms bounce 0px -4.43144px spin 270deg   slide 139.334px | fill none=#796538, forwards=#796538,
backwards=#796538, both=#796538
t=1000ms bounce 0px -12px spin 0deg     slide 160px     | fill none=grey (own style), forwards=green,
backwards=grey (own style), both=green
t=1500ms bounce 0px 12px  spin 180deg   slide 80px      | fill none=grey (own style), forwards=green,
backwards=grey (own style), both=green
events on .fm.both (restarted): animationstart after ~500ms, animationend after ~1000ms
```

- **Bounce at 250ms**: the 0 % → 50 % segment lasts 500ms with `ease-in`; ease-in(0.5) = 0.315; −12 + 24 × 0.315 = **−4.43px** ✓. At 750ms the 50 % → 100 % segment uses `ease-out`: 12 − 24 × 0.685 = **−4.43px** ✓ — per-keyframe easing.
- **Slide** (`ease-in-out`, alternate): ease-in-out(0.25) = 0.129 × 160 = **20.67px** ✓; at 1500ms the second iteration runs backwards, half-way → **80px** ✓.

- **fill-mode** — the most misunderstood property (e.g. a fade-in that's visible during its delay needs `backwards` or `both`):

Mode	During the delay (0–500ms)	After the end (≥ 1000ms)
`none`	element's own style (grey)	own style (grey)
`forwards`	own style	**last keyframe** (green)
`backwards`	**first keyframe** (red)	own style
`both`	first keyframe	last keyframe

- At 750ms all four show **#796538**, the halfway mix of #dc2626 and #16a34a in sRGB (legacy `rgb`/hex colours interpolate in sRGB): (220 + 22)/2 = 121 = 0x79, (38 + 163)/2 ≈ 101 = 0x65, (38 + 74)/2 = 56 = 0x38.

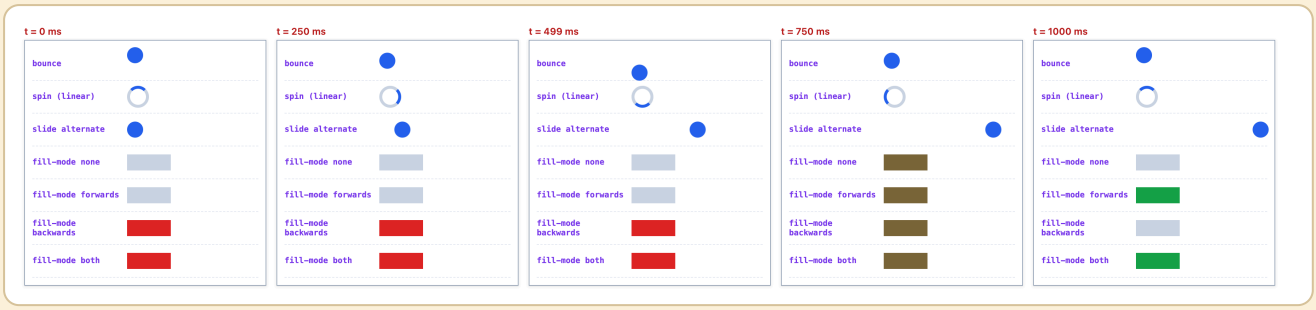


Figure 12.5 — Filmstrip of `code/05_keyframes.html`.

Events

`animationstart` (after the delay), `animationiteration` (each loop), `animationend`, `animationcancel` — the output's last line shows start at ~500ms and end at ~1000ms.

Transition or animation?

Use a **transition** when…	Use a **keyframe animation** when…
Animating between two states triggered by a change (hover, class, open/closed)	It must run by itself (on load, looping, attention-seeking)
Only a start and an end	You need intermediate steps
It should reverse naturally when the state flips back	You need iteration, direction, delay choreography

---

12.6 The Web Animations API (WAAPI)

What it is

The **Web Animations API** is JavaScript's interface to the same engine that runs CSS transitions and animations (Baseline widely available since 2020–2023). `element.animate(keyframes, options)` creates one; every animation, CSS or JS, is an `Animation` object you can pause, seek, speed up, reverse and `await`.

```
const anim = box.animate(
  [{ translate: '0 0', opacity: 1 }, { translate: '200px 0', opacity: 0.5 }],   // keyframes
  { duration: 400, easing: 'ease-out', fill: 'forwards' });                      // options
anim.pause(); anim.currentTime = 200;        // scrub
anim.playbackRate = 2; anim.play();          // double speed
await anim.finished;                         // a Promise
anim.reverse();
document.getAnimations();                    // every running animation, CSS ones included
```

Output of `code/11_waapi.mjs` (page: `code/11_waapi.html`):

```
created: playState=running, listed by document.getAnimations(): true
paused at 200ms: translate=136.929px, opacity=0.658
finished: currentTime=400ms (the remaining 200ms took ~100ms of real time at 2x), playState=finished
fill: forwards keeps the end: translate=200px
after reverse(): translate=none  (back at the start; fill: forwards doesn't hold the START value)
```

ease-out(0.5) = 0.685 → 200 × 0.685 = 136.9px; opacity 1 − 0.5 × 0.685 = 0.658 ✓.

When to use it

- Values known only at runtime (animating from a clicked element's position — FLIP, 12.7).
- Sequencing with promises (`await a.finished; b.play()`), scrubbing with a slider, syncing with media.
- Testing and screenshots — how this chapter's figures were made.

Prefer CSS for simple **declarative** states; prefer WAAPI over hand-written `requestAnimationFrame` loops, because WAAPI `transform`/`opacity` animations can run on the compositor (12.7) and are cancellable and inspectable in DevTools' Animations panel. `anim.commitStyles()` writes the current values into the `style` attribute — useful before `cancel()` to keep the end state without `fill: forwards` (fills keep an animation alive forever).

12.7 Performance: animate `transform` and `opacity`

How the browser renders a frame

Each frame re-runs the rendering pipeline — **style → layout → paint → composite** (§1.7) — from the first stage the changed property affects. Style, layout and paint run on the **main thread** (which also runs your JavaScript); compositing can run on a separate **compositor thread**, often on the GPU.

Property changed	Needs	Examples	Cost
Geometry	style → **layout** → paint → composite	`width`, `height`, `top`/`left`, `margin`, `padding`, `font-size`	Highest: the element *and possibly everything around it* is re-laid out
Appearance	style → **paint** → composite	`color`, `background`, `box-shadow`, `border-radius`, `outline`	Medium: pixels re-drawn
Compositor-only	(style) → **composite**	`transform` (and `translate`/`rotate`/`scale`), `opacity`, often `filter`	Lowest: the GPU moves/fades an already-painted layer, even while the main thread is busy

At 60 fps a frame has about **16.7ms**; on a 120 Hz screen, 8.3ms. Layout of a big page can take several ms alone, so layout animations drop frames (jank).

Measured

`code/06_performance.mjs` runs four 1-second animations of the same box in `code/06_performance.html` and reads Chrome's counters (`page.metrics()`; a "style recalc" is one run of the style stage):

```
move-left       layouts  60   style recalcs  61   layout time 1.8ms
grow-width      layouts  60   style recalcs  61   layout time 1.8ms
move-transform  layouts   1   style recalcs   5   layout time 0.0ms
fade            layouts   1   style recalcs   5   layout time 0.0ms
```

`left`/`width` ran layout on **every frame** (~60 in one second); `transform` and `opacity` needed **one** in total — the compositor animated them off the main thread. (Counts vary ±1 per run; layout time is tiny only because the page is tiny — on a real page each of those 60 layouts could cost milliseconds.)

Techniques

- **Move with `translate`, not `top`/`left`; resize with `scale`, not `width`/`height`; fade with `opacity`.**
- **Expensive paints**: animate the `opacity` of a pseudo-element that already has the big shadow, rather than `box-shadow`.
- **FLIP** (First, Last, Invert, Play) for layout changes (e.g. a card expanding from a grid into a detail view): measure the **F**irst position, apply the layout change, measure the **L**ast, **I**nvert with a transform so it *looks* unmoved, then **P**lay a transform animation back to none. Layout happens once; the motion is a cheap transform. View transitions (12.10) automate this.
- `will-change: transform` hints the browser to promote the element to its own **layer** (a separately painted image the compositor can move without repainting) ahead of time. Sparingly and temporarily: every layer costs GPU memory, and `will-change` creates a stacking context.
- `contain: layout paint` / `content-visibility` limit how far layout and paint spread (Chapter 14).
- **Measure**: DevTools Performance panel (purple Layout and green Paint blocks per frame), Rendering → "Paint flashing" and "Layer borders", and the Animations panel to slow animations down.

In the interview

30-second answer: "Each frame the browser does style, layout, paint and composite. Animating layout properties like `left` or `width` re-runs layout and paint every frame — I measured about 60 layouts per second for a `left` animation. `transform` and `opacity` can be handled by the compositor on the GPU, so I animate those — translate instead of top/left, scale instead of width — and use FLIP when a layout change must look animated. `will-change` sparingly, and I check the Performance panel for long frames."

- *Is `filter` compositor-only?* → Often in Chromium, but a blur can be expensive; test.
- *Why can a transform animation keep running while JS is busy?* → It runs on the compositor thread.
- *Downside of `will-change` everywhere?* → Too many layers → memory pressure and slower pages; also stacking contexts.

12.8 Enter and exit animations: `@starting-style` and `allow-discrete`

The problem it solves

Dialogs, popovers (floating panels opened with the `popover` attribute), toasts (brief self-dismissing notices) and menus usually go from `display: none` to visible, which breaks transitions twice:

1. **Entering**: an element that wasn't rendered has no "before" style — it just appears.
2. **Exiting**: `display` flips to `none` immediately, so the fade-out is never seen.

The tools

- **`@starting-style { … }`** defines the style to transition **from** when an element is first rendered (or leaves `display: none`). Baseline since August 2024.
- **`transition-behavior: allow-discrete`** (or the keyword in the shorthand) lets discrete properties join a transition; for `display`, the element stays displayed for the whole exit and flips to `none` at the end. `transition-behavior` is Baseline since August 2024, but transitioning `display` works in Chrome and Safari, not yet Firefox, and the `overlay` property (keeps a closing popover/dialog in the **top layer**, drawn above everything) is Chromium-only. Where unsupported the element simply closes without the fade — safe progressive enhancement.

```
#a { opacity: 0; translate: 0 -16px;                          /* closed look = exit end state */
     transition: opacity 300ms ease-out, translate 300ms ease-out,
                 display 300ms allow-discrete, overlay 300ms allow-discrete; }
#a:popover-open { opacity: 1; translate: 0 0; }               /* open look */
@starting-style { #a:popover-open { opacity: 0; translate: 0 -16px; } }   /* enter starts here */
```

`code/07_starting_style.html` has popover **A** (above) and **B** (same transition without `@starting-style` or `allow-discrete`). `code/07_starting_style.mjs` opens and closes both, freezing at 100ms:

```
opening, 100ms: A: display block opacity 0.49  transitions running: opacity+translate   |   B: display block
opacity 1.00  transitions running: none
opened        : A: display block opacity 1.00  transitions running: none   |   B: display block opacity 1.00
transitions running: none
closing, 100ms: A: display block opacity 0.51  transitions running: display+opacity+overlay+translate   |   B:
display none  opacity 0.00  transitions running: none
closed        : A: display none  opacity 0.00  transitions running: none   |   B: display none  opacity 0.00
transitions running: none
```

B never animates (no starting style on open; `display: none` immediately on close). A fades in from the `@starting-style` values and, while closing, stays `display: block` with a running `display` transition until the fade ends.

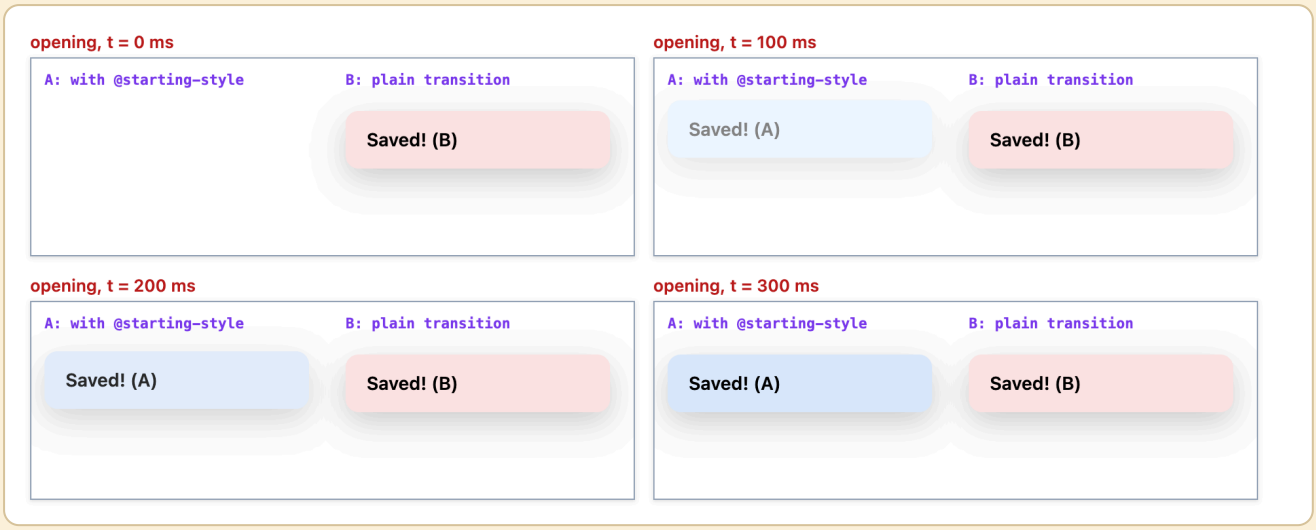


Figure 12.6 — A (blue) animates in; B (red) just appears.

Where you'll use it

`<dialog>` (`[open]` and its `::backdrop`, transitioned the same way), `[popover]` (`:popover-open`), JS-added elements (toasts, list items), `<details>` content, and anything toggled with `display` or `hidden`.

12.9 Reduced motion and accessibility

Why it matters

For people with **vestibular disorders** (inner-ear balance problems), large on-screen movement — parallax, zooming, spinning, sliding panels — can trigger dizziness, nausea and headaches; others find motion distracting (ADHD, cognitive disabilities). Every major OS has a setting (macOS/iOS "Reduce motion", Windows "Animation effects" off, Android "Remove animations"), exposed as `prefers-reduced-motion: reduce` (Baseline widely available).

The WCAG rules

(Levels: A minimum, AA the usual legal target, AAA strictest — §3.9.)

Criterion	Level	Requirement
2.2.2 Pause, Stop, Hide	A	Auto-starting moving, blinking or scrolling content lasting **more than 5 seconds** alongside other content must be pausable/stoppable (carousels, marquees, animated backgrounds, a 30s looping hero video)
2.3.1 Three Flashes or Below Threshold	A	Nothing flashes more than **3 times per second** (seizure risk)
2.3.3 Animation from Interactions	AAA	Interaction-triggered motion can be disabled unless essential

Two patterns

`code/08_reduced_motion.html`:

```
/* 1. Motion only when allowed: big movement is ADDED for users with no preference */
@keyframes slide-in { from { translate: -60px 0; opacity: 0; } }
@media (prefers-reduced-motion: no-preference) {
  .card { animation: slide-in 600ms ease-out both; }
  html  { scroll-behavior: smooth; }
}

/* 2. Replace, don't remove: a loading spinner must still say "busy" */
@keyframes pulse { 50% { opacity: .35; } }
.spinner { animation: spin 800ms linear infinite; }
@media (prefers-reduced-motion: reduce) { .spinner { animation: pulse 2s ease-in-out infinite; } }
```

`code/08_reduced_motion.mjs` emulates both settings:

```
prefers-reduced-motion: no-preference -> card animation: slide-in spinner: spin 0.8s   scroll-behavior: smooth
running animations: 2
prefers-reduced-motion: reduce        -> card animation: none     spinner: pulse 2s   scroll-behavior: auto
running animations: 1
```

Reduced motion ≠ no motion. Fades, colour changes and small scale changes are usually fine; large translation, zoom, rotation, **parallax** (layers moving at different scroll speeds) and **scroll-jacking** (overriding normal scroll speed or direction) cause harm. The blunt global reset in §10.5 (durations → `0.01ms`, not `0`, so `animationend` / `transitionend` still fire for code that waits on them) is a reasonable safety net, but designing each animation's reduced version is better. In JS, check `matchMedia('(prefers-reduced-motion: reduce)').matches` before WAAPI animations, smooth `scrollIntoView`, or autoplaying video.

Other rules of thumb

- Autoplaying carousels need a visible pause button (2.2.2) — and usually shouldn't autoplay.
- Don't put essential information only in motion.
- Keep focus behaviour intact: don't move focus into an element that's still invisible.

12.10 View transitions

What it is

A **view transition** animates between two **states of the page** — before and after a DOM update, or (cross-document) between pages. The browser **snapshots** the old state, lets you update the DOM, snapshots the new state, and animates — cross-fading the page and **morphing** each element with a `view-transition-name` (captured separately, it moves from its old size/position to the new one). It's FLIP (12.7) done for you.

Support (verified September 2026)

Feature	Status
Same-document: `document.startViewTransition()`, `view-transition-name`, the `::view-transition-*` pseudo-elements	**Baseline newly available since October 2025** (Chrome 111, Safari 18, Firefox 144)
`view-transition-class` (style many groups with one selector)	Baseline since October 2025
View transition **types** (`startViewTransition({ update, types: ['forward'] })`, `:active-view-transition-type()`)	Baseline since January 2026
Cross-document (MPA — multi-page application) view transitions: `@view-transition { navigation: auto; }`	**Not Baseline**: Chrome 126+, Safari 18.2+, not Firefox yet

How it works (same-document)

1. You call `document.startViewTransition(updateCallback)`.
2. The browser captures the **old** state: the page (`root`) and each element with a (unique) `view-transition-name`.
3. It runs your callback (may be async) to change the DOM, then captures the **new** state.
4. It builds a pseudo-element tree over the page:

```
::view-transition
└─ ::view-transition-group(item-2)        ← animates size and position old → new
   └─ ::view-transition-image-pair(item-2)
      ├─ ::view-transition-old(item-2)     ← old snapshot, fades out
      └─ ::view-transition-new(item-2)     ← new live view, fades in
```

5. It animates the tree with ordinary CSS animations (default 250ms `ease`), then removes it; the page underneath already shows the new state. Style `::view-transition-group(name)` (or `(*.class)`) to change the morph.

```
const update = () => items.classList.toggle('list');
toggle.addEventListener('click', () => {
  if (!document.startViewTransition) return update();     // no support: just update
  document.startViewTransition(update);
});
```

```
.item:nth-child(1) { view-transition-name: item-1; }       /* one unique name per morphing element */
.item { view-transition-class: item; }
::view-transition-group(*.item) { animation-duration: 500ms; animation-timing-function: ease-in-out; }
::view-transition-old(*.item), ::view-transition-new(*.item) { height: 100%; }   /* see below */
@media (prefers-reduced-motion: reduce) {
  ::view-transition-group(*), ::view-transition-old(*), ::view-transition-new(*) { animation: none !important; }
}
```

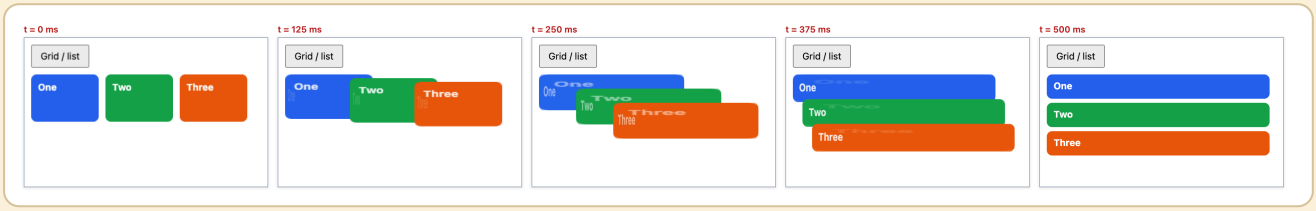


Figure 12.7 — `code/09_view_transition.html` going grid → list, frozen at five moments via `document.getAnimations()`.

`code/09_view_transition.mjs` waits for `transition.ready` (pseudo-elements exist), pauses everything and inspects it (abridged — item-2 and item-3 repeat item-1's animations):

```
20 animations while the transition runs:
  ::view-transition-group(item-1) -ua-view-transition-group-anim-item-1 500ms
  ::view-transition-group(root) -ua-view-transition-group-anim-root 250ms
  ::view-transition-new(item-1) -ua-mix-blend-mode-plus-lighter 500ms
  ::view-transition-new(item-1) -ua-view-transition-fade-in 500ms
  ::view-transition-old(item-1) -ua-view-transition-fade-out 500ms
  … (plus mix-blend-mode-plus-lighter on old(item-1), the root old/new pairs at 250ms, and item-2/item-3)
t=  0ms ::view-transition-group(item-2): width 100.0px, height 70.0px, transform matrix(1, 0, 0, 1, 120, 54)
t=250ms ::view-transition-group(item-2): width 215.0px, height 53.0px, transform matrix(1, 0, 0, 1, 65, 75)
t=500ms ::view-transition-group(item-2): width 330.0px, height 36.0px, transform matrix(1, 0, 0, 1, 10, 96)
```

`-ua-…` are Chrome's built-in keyframes. "Two" morphs from 100×70 at (120, 54) to a 330×36 row at (10, 96); at half-way (`ease-in-out`(0.5) = 0.5) every value is the average — 215 = (100 + 330)/2 ✓. `root` keeps 250ms because the rule only targeted `*.item`.

Aspect-ratio tip: old and new snapshots keep their own aspect ratios at the group's width, so a tall box becoming a short row looks smeared mid-way; `height: 100%` on `::view-transition-old/new` stretches both to the group (used in Figure 12.7).

Cross-document view transitions

Both pages opt in with `@view-transition { navigation: auto; }` and share `view-transition-name`s; a **same-origin** navigation then animates with no JavaScript. Chrome and Safari only in 2026 — others just navigate, so it's safe to add.

Pitfalls

- Duplicate `view-transition-name`s abort the transition.
- During a transition the overlay blocks clicks — keep it short.
- Snapshots are images: text stretches during a morph; cross-fade handles most cases.
- Keep the no-support path (`if (!document.startViewTransition) return update();`).

12.11 Scroll-driven animations

What it is

A **scroll-driven animation** replaces time with a **scroll timeline**: progress follows how far a container has scrolled (`scroll()`) or how far an element has travelled through the viewport (`view()`). Reading-progress bars, reveal-on-scroll and parallax without JS scroll listeners — and potentially off the main thread.

Support: not Baseline. Chrome 115+ and Safari 26+; Firefox only behind a flag (September 2026). Always wrap in `@supports (animation-timeline: scroll())` (a feature query) and make the page work without it.

```css
/* Reading progress bar */
@keyframes grow { from { scale: 0 1; } to { scale: 1 1; } }
.progress { transform-origin: 0 50%; animation: grow linear both; animation-timeline: scroll(root block); }

/* Reveal an element as it enters the viewport */
@keyframes fade-up { from { opacity: 0; translate: 0 40px; } to { opacity: 1; translate: 0 0; } }
.reveal { animation: fade-up linear both; animation-timeline: view(); animation-range: entry 0% entry 100%; }
```

Piece	Meaning
`scroll(<scroller> <axis>)`	Anonymous scroll timeline: `nearest` (default), `root` or `self`; axis `block` (default), `inline`, `x`, `y`
`view(<axis> <inset>)`	Timeline of the element's own visibility in its scrollport
`animation-range`	Which part of the timeline maps to 0–100 %: `entry`, `exit`, `contain`, `cover` + percentages
`scroll-timeline-name` / `view-timeline-name`	Named timelines, usable by other elements (with `timeline-scope`)
Duration	Ignored/`auto` — scroll distance replaces time; use `linear` easing so progress tracks scroll 1:1

`code/10_scroll_driven.mjs` scrolls `code/10_scroll_driven.html` (400px-tall viewport) and reads the progress bar and the second reveal box:

```
CSS.supports("animation-timeline: scroll()") = true
scrollY   0px (  0% of 540px): bar scale 0 1       #r2 top  562px (height 48) opacity 0.00 translate 0px 40px
scrollY 135px ( 25% of 540px): bar scale 0.25 1    #r2 top  427px (height 48) opacity 0.00 translate 0px 40px
scrollY 186px ( 34% of 540px): bar scale 0.344444 1 #r2 top  376px (height 48) opacity 0.50 translate 0px
20.1042px
scrollY 270px ( 50% of 540px): bar scale 0.5 1     #r2 top  292px (height 48) opacity 1.00 translate 0px
scrollY 540px (100% of 540px): bar scale 1         #r2 top   22px (height 48) opacity 1.00 translate 0px
```

The bar's scale equals the scroll fraction (186/540 = 0.344). The box's `entry` range runs from "top touches the viewport bottom" (top = 400px) to "fully inside" (400 − 48 = 352px); at 376px it's half-way → opacity ≈ 0.50, translate ≈ 20px ✓.

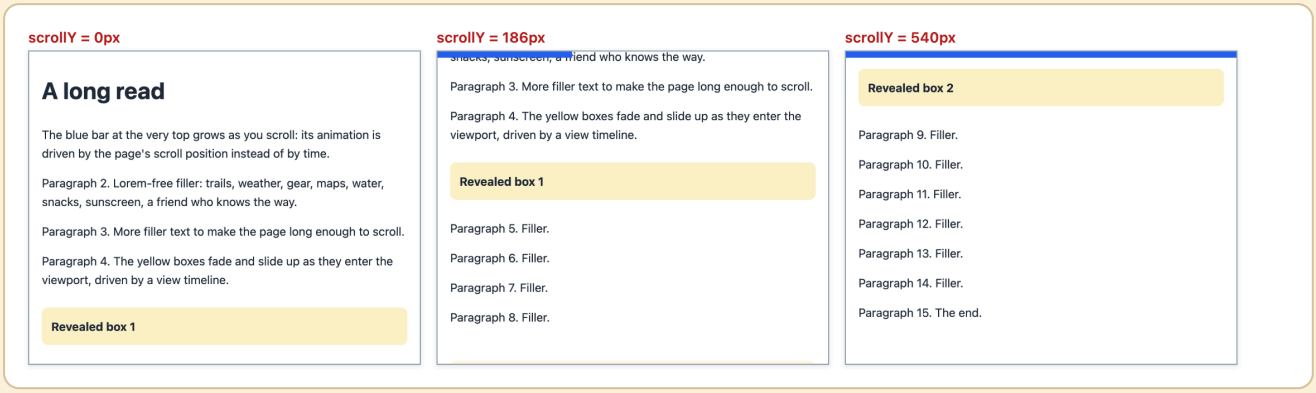


Figure 12.8 — The progress bar at three scroll positions.

Cautions

- Respect reduced motion: big parallax or slide-ins inside `@media (prefers-reduced-motion: no-preference)` (the demo disables the reveal under `reduce`).
- Don't hide content until it scrolls in if it might never animate (unsupported browser, print, find-in-page); the demo's `@supports` wrapper keeps content visible by default.
- Scroll-driven animations should *follow* scrolling, never replace it (no scroll-jacking).

Chapter 12 TL;DR

- **Transitions** animate a *change*: `transition: prop duration easing delay`, declared on the base rule; interpolable values only (not `auto`); `transitionend` per property; reversal is shortened.
- **Timing functions**: `ease` (default), `ease-in` (leaving), `ease-out` (entering), `ease-in-out`, `cubic-bezier()` (x ∈ 0–1, y may overshoot), `steps(n)` (jump-end by default), `linear(points)` (Baseline 2023). Our bisection solver matched Chrome to 3 decimals.

- **2D transforms** move the *painted* box, not the layout box (layout stayed 50,30 90×50 for all eight). Order matters; `transform-origin`; `%` translate = own size; individual `translate`/`rotate`/`scale` (Baseline 2022) apply in fixed order and animate independently. Transforms create stacking contexts and containing blocks.
- **3D**: `perspective` on the parent, `preserve-3d`, `backface-visibility: hidden` → flip cards; `overflow`, `opacity < 1`, `filter` flatten 3D.
- **Keyframes**: `@keyframes` + `animation: name duration easing delay count direction fill-mode`; easing per segment; **fill-mode** = what shows during the delay (backwards) and after the end (forwards); negative delays start mid-way.
- **WAAPI**: `el.animate()`, `pause()`, `currentTime`, `playbackRate`, `reverse()`, `await anim.finished`, `document.getAnimations()` (includes CSS animations).
- **Performance**: animate `transform` and `opacity` (≈1 layout per animation), not `left`/`width` (≈60 layouts/second, measured). FLIP for layout changes; `will-change` sparingly.
- **Enter/exit**: `@starting-style` (Baseline 2024) for the "from" style; `transition-behavior: allow-discrete` + `display`/`overlay` keep an element visible while animating out (display: Chrome/Safari; overlay: Chromium).
- **Reduced motion**: `@media (prefers-reduced-motion: no-preference)` to opt motion in; replace rather than remove; WCAG 2.2.2 (pause > 5s auto motion), 2.3.1 (≤ 3 flashes/s), 2.3.3 (AAA).
- **View transitions**: `document.startViewTransition(update)` + `view-transition-name` → snapshot, update, morph via `::view-transition-group/old/new`; same-document Baseline since Oct 2025; cross-document Chrome/Safari only.
- **Scroll-driven**: `animation-timeline: scroll()` / `view()` + `animation-range`; **not Baseline** (no Firefox) → `@supports`, content visible without it.

Next: Chapter 13 — Modern CSS and Architecture.